Features and Representations

A feature is an input variable representing some aspect of an observation. Raw data may not be directly useful to an analysis, so feature extraction transforms information into a representation that captures relevant patterns. A useful feature should have a clear definition, be available at the right time, and relate to the analytical task.

Feature extraction differs from feature selection. Extraction creates new representations or transforms raw inputs; selection chooses a subset of existing inputs. Both require domain understanding and validation. More features do not always produce better results and can increase complexity, noise, and fairness risk.

Key points

- Define each feature's meaning, units, and source.
- Consider whether the feature would be available at prediction time.
- Keep a simple baseline for comparison.

Cleaning, Encoding, and Scaling

Missing values and unusual observations must be investigated before transformations are applied. Numeric features may be scaled so variables with large units do not dominate distance-based methods. Standardization centers and scales values, while min-max scaling maps values into a range; the appropriate choice depends on the method and data.

Categorical variables can be represented using one-hot encoding or other schemes. An ordinal encoding should only be used when category order has meaning. Every transformation should be documented because the resulting numeric representation may not preserve all original distinctions.

Key points

- Choose an imputation strategy based on why values are missing.
- Encode categories without inventing a false order.
- Fit scaling and imputation on training data only.

Dates, Text, and Domain Features

Dates and times can yield features such as weekday, month, elapsed time, or time since an event. These features should match the problem and avoid using information from the future. Text may be represented through token counts, term frequency, or TF-IDF, but preprocessing decisions can affect meaning and fairness.

Domain features combine subject knowledge with observed fields—for example, a rate may be more meaningful than a raw count when exposure differs. Write down the formula, units, and assumptions. Carefully check edge cases such as zero denominators, time zones, spelling variations, and rare categories.

Key points

- Create features that can be explained and reproduced.
- Avoid features that leak the outcome or future information.
- Inspect distributions and examples before relying on a transformation.

Dimensionality Reduction

When many features are correlated or high-dimensional, dimensionality reduction can create a smaller representation. Principal component analysis (PCA) finds directions that capture variation in numeric data. Components are combinations of original variables and may be harder to interpret than the source features.

PCA is sensitive to scale, so standardization is often considered first when units differ. The number of components should be selected for a reason, and explained variance does not guarantee that the representation preserves information relevant to a particular prediction task.

Key points

- Standardize when scale differences would dominate the analysis.
- Fit PCA only on training data in predictive workflows.
- Explain the interpretability trade-off and information loss.

Pipelines and Data Leakage

Data leakage happens when information unavailable at the intended decision time influences training or evaluation. Leakage can make performance appear much better than it will be in real use. Common causes include fitting a scaler on the full dataset, using future outcomes in a feature, or allowing the same person to appear in training and test sets.

A preprocessing pipeline applies transformations in a consistent order. Split data before learning parameters such as means, category mappings, or PCA components. During cross-validation, fit the full pipeline independently within each training fold so evaluation remains honest.

Key points

- Split before fitting learned preprocessing steps.
- Use group- or time-aware splits when records are dependent.
- Keep a final test set untouched until design decisions are complete.

Feature Engineering Project

Select a small dataset and define a target or analysis goal. Document at least three candidate features, including their source fields, formulas, units, and expected availability. Establish a simple baseline before adding transformations so their contribution can be assessed fairly.

Create a train/test split appropriate to the data, fit preprocessing only on training observations, and compare results with the baseline. Inspect errors and subgroup performance, note information lost through transformations, and describe potential leakage or fairness issues.

Key points

- Deliver: feature dictionary, reproducible preprocessing workflow, baseline comparison, and limitations.
- Review: How does extraction differ from selection? Why fit scalers only on training data?
- Report whether the features improve the task and remain meaningful for intended use.